

Sampling in Particle-Filter Localization and SLAM

Mohammed

September 2026

Summary

Localization and SLAM are estimation problems on a graphical model whose structure is fixed in advance by how the robot moves and what its sensor reads, so the graph itself never has to be learned. What the problem does leave open is the inference: the belief over the robot's poses cannot be computed exactly for this model, so it is approximated by sampling. The noise of the motion and measurement models is almost always set by hand; in this project I estimate it from the data the robot recorded.

I built a testbed that makes a particle filter's sampling choices interchangeable, so that one can be varied while the world, the trajectory and the random seed stay fixed, and measured two of them over twenty simulated worlds in localization and in SLAM: which resampling scheme is used, and when resampling happens. Whether the filter resamples at all is what matters; among the rules that do resample, and among the four schemes, accuracy is the same. The schemes differ instead in how many distinct particles survive, and that reaches the estimate only in FastSLAM with few particles, where a lost particle takes its map with it.

The weights the filter already computes also score how well a given noise setting explains the recording, so the noise can be estimated rather than assumed. The two measurement parameters come back close to the values the world used, 0.19 m against 0.2 m and 0.047 rad against 0.05 rad. The two motion parameters leave no trace in a 30-step run and are recovered on a 400-step one. Learning them at all needs the filter to be tracking, and enough particles for the score to be reliable, so how good the inference is sets a ceiling on what can be learned.

1 Background

A robot that must know where it is has two unreliable sources of information: the controls it was given, which are corrupted by wheel slip, and the measurements it takes of its surroundings, which are corrupted by sensor noise. Combining them into an estimate of the pose, with an honest account of its uncertainty, is probabilistic localization; doing it while building the map is SLAM [1]. This section gives what is needed to follow the rest of the report: the network the problem is written on, how inference is done on it exactly and how it has to be approximated, what makes that approximation affordable in SLAM, and finally what it means to learn the model's numbers rather than supply them.

Localization and SLAM as a dynamic Bayesian network

In localization the map is known and the robot estimates its pose from the controls it was given and its observations of known landmarks. In SLAM the map is unknown, and the robot estimates the map and its own trajectory at the same time.

Both are naturally written as a dynamic Bayesian network [2, ch. 6 and 15]. The pose $\mathbf{x}_t = (x_t, y_t, \theta_t)$ depends on the previous pose and the control $\mathbf{u}_t$ through the motion model

$p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{u}_t)$; each observation $\mathbf{z}_t$ is conditioned on the pose from which it was taken and on the map $\mathbf{m}$ through the measurement model $p(\mathbf{z}_t | \mathbf{x}_t, \mathbf{m})$ (Fig. 1), so

$$p(\mathbf{x}_{0:T}, \mathbf{m}, \mathbf{z}_{1:T} | \mathbf{u}_{1:T}) = p(\mathbf{x}_0) p(\mathbf{m}) \prod_{t=1}^T p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{u}_t) p(\mathbf{z}_t | \mathbf{x}_t, \mathbf{m}). \quad (1)$$

Controls and observations are observed; the trajectory is latent, and in SLAM the map is latent as well. Correspondences are known throughout, so each observation carries the identity of the landmark it measures.

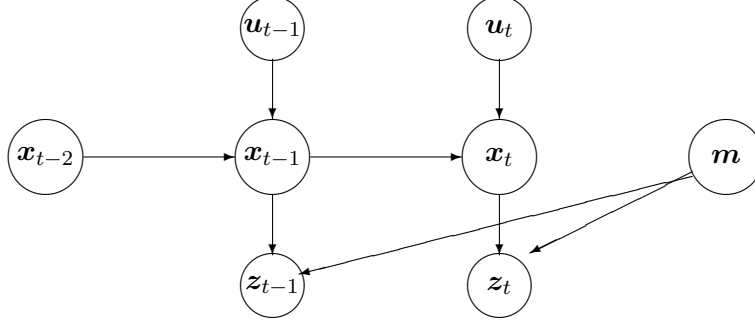


Figure 1: Two time slices of the network. Controls $\mathbf{u}$ and observations $\mathbf{z}$ are observed; poses $\mathbf{x}$ and the map $\mathbf{m}$ are latent. In localization the map is known.

Exact inference on the network: the Bayes filter

Estimation on this network is the Bayes filter, the forward pass of inference on the network written out over time. What it computes is the *belief*, the distribution over the current pose given everything observed so far. Because $\mathbf{x}_t$ separates past from future, the belief satisfies

$$p(\mathbf{x}_t | \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \propto p(\mathbf{z}_t | \mathbf{x}_t) \int p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{u}_t) p(\mathbf{x}_{t-1} | \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t-1}) d\mathbf{x}_{t-1}, \quad (2)$$

a prediction through the motion model followed by a correction with the new observation [1].

The state is continuous, so the belief is a density and the prediction is an integral. That integral is tractable in two cases: with a discrete state space it is a finite sum, and (2) is the forward algorithm of a hidden Markov model; with a linear model and Gaussian noise the belief stays Gaussian and the recursion is the Kalman filter. Range and bearing measurements are nonlinear in the pose, and under global uncertainty the belief is multimodal, so neither applies and no closed form exists.

Approximate inference: the particle filter

Where the integral cannot be carried out it is replaced by samples. The distribution the samples are drawn from is the *proposal*. Drawing a pose from the motion model and weighting it by $p(\mathbf{z}_t | \mathbf{x}_t)$ is likelihood weighting applied to one slice, with the prior as the proposal [2, ch. 12]; carrying the weighted set forward to the next slice makes it sequential [3], [4]. The belief after step t is N particles $\mathbf{x}_t^{(i)}$ with normalized weights $w_t^{(i)}$.

One ingredient has to be added. Over time the weights become products of more and more terms, so they concentrate on a single particle and the set stops representing the belief; how far this has gone is measured by the effective sample size $\text{ESS} = 1 / \sum_i (w^{(i)})^2$. Resampling, which replaces the set by N draws from itself, prevents it, and has no counterpart in the exact algorithm. It costs something in return: drawing with replacement duplicates the heaviest particles and discards the rest, so the number of distinct particles falls. A filter therefore has four things to decide: the proposal, when to resample, how to resample, and how many particles to carry. The middle two are the subject of this report.

Rao-Blackwellization and FastSLAM

Applying the particle filter to SLAM raises a problem of dimension. The state is now the pose together with the map, so each particle would have to carry a complete map as well as a pose, and the state space gains two dimensions per landmark. Sampling that joint space directly is intractable: the number of particles needed to cover it grows exponentially with the number of landmarks.

The structure of the network offers a way out. An observation of landmark $\mathbf{m}_k$ has exactly two parents: the pose it was taken from and $\mathbf{m}_k$ itself. Any path between two landmarks must therefore leave one of them through an observation and pass through that observation's pose, with both edges pointing away from the pose, and a known trajectory blocks every such path. The landmarks are d-separated given the trajectory and the observations [2], so the posterior splits:

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{1:t} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \prod_{k=1}^K p(\mathbf{m}_k \mid \mathbf{x}_{1:t}, \mathbf{z}_{1:t}). \quad (3)$$

The first factor is a distribution over trajectories, the hard part. Each of the others is a small problem over one landmark's two coordinates, given that trajectory. Splitting a posterior in this way and solving the easy part exactly, rather than by sampling it, is Rao-Blackwellization [5], and FastSLAM is its application here: every particle carries one trajectory and one extended Kalman filter per landmark [6]. Only the trajectory is sampled, which is what makes particle-filter SLAM affordable.

Learning the parameters of the network

Everything above takes the model as fully specified and asks how to compute with it. A graphical model raises a second question: where its numbers come from.

The structure of (1) fixes which distributions appear, not the numbers inside them. Write those numbers as θ . They may be supplied, or estimated from data; estimating them is *parameter learning*, as opposed to structure learning, which asks for the graph itself.

When some of the variables are never observed it is parameter learning *with incomplete data* [2, ch. 19], and that is the case here: the trajectory the observations depend on is latent, so the only data are the controls and the observations. A dynamic Bayesian network is also a template model, so the same conditional distribution is instantiated at every time slice and its parameters are tied across slices [2, ch. 6]. Every step is then evidence about the same θ , and a longer trajectory is more data about it rather than more unknowns.

The estimate is the θ under which the observations are most probable,

$$\hat{\theta} = \arg \max_{\theta} \log p(\mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}, \theta), \quad p(\mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}, \theta) = \prod_{t=1}^T p(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}, \theta). \quad (4)$$

Its maximum is in the right place and not merely a convenient one: the average log likelihood tends to a term free of θ minus the Kullback-Leibler divergence between the true distribution and the model, and that divergence vanishes only where the two agree [2, ch. 17].

Each factor of (4) is again an integral with no closed form, and the particle filter estimates it as a by-product. With particles propagated through the motion model the unnormalized weight is $\tilde{w}_t^{(i)} = w_{t-1}^{(i)} p(\mathbf{z}_t \mid \mathbf{x}_t^{(i)}, \theta)$, and

$$p(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}, \theta) \approx \sum_{i=1}^N \tilde{w}_t^{(i)}, \quad (5)$$

the sum the filter forms anyway in order to normalize [7].

Two consequences follow, and both appear in the results. The estimate of the likelihood is unbiased, so the estimate of its logarithm is biased downwards, and the bias grows as the particle approximation degrades: the quality of the inference limits what can be learned. And the curvature of the expected score at θ^* is $T\mathcal{I}(\theta^*)$, with $\mathcal{I}$ the information one step carries, so the variance of the estimate is $1/(T\mathcal{I})$. A parameter the observations say little about leaves a flat likelihood: it is in the model and still not identifiable from the data, and because the parameters are tied, a longer trajectory sharpens it.

2 Problem statement and objectives

The structure of the model is fixed in advance by the robot’s kinematics and its sensor geometry, so there is no structure to learn. What is left is the inference: the belief cannot be computed in closed form, so it is approximated by a set of samples, and the estimate is only as good as that set. How that set is proposed, weighted and resampled is the first thing measured here.

The motion model and the measurement model each carry a noise level, and in practice these are almost always set by hand. In this project I estimate them instead, from the data the robot itself produced. That is the second thing measured here, and the weights the filter already computes are what make it possible.

Sampling. A filter must choose a proposal distribution, a rule for when to resample, a resampling scheme and a number of particles, and these are usually inherited from whatever implementation is at hand. I built a testbed in which they are interchangeable parts of one filter loop, so that one can be varied while the world, the trajectory and the random seed stay fixed. The filters themselves are existing open-source implementations: the localization filters and resampling schemes of Elfring et al. [8], and FastSLAM 1.0 from PythonRobotics [9].

Two of the four are the subject of this report:

- *the resampling scheme*: multinomial, residual, stratified and systematic resampling [10], [11], which differ in the variance they add when the set is redrawn;
- *when to resample*: at every step; never; when the effective sample size falls below a threshold (0.2, 0.5 or 0.8 of N); or when the largest weight rises above one (0.1, 0.2 or 0.5).

The other two are held fixed so that they cannot confound the comparison. The proposal is always the motion model, which draws from the prior, and the number of particles is fixed within each comparison.

Parameter learning. The noise values are parameters of the network’s two conditional distributions, and the weights the filter already computes estimate the marginal likelihood of the observations under them, so they can be estimated from a recorded run instead of assumed. This asks not how well the filter infers under a given model, but which parts of the model the data can determine at all. What is tested:

- whether the four noise parameters can be recovered by maximizing (4), and how the recovered values compare with the values the world used;
- which of them the data identify: whether the likelihood has a maximum, and whether independent recordings agree on where it is;
- what the inference must be capable of first: how the estimate depends on the number of particles, on the resampling scheme, and on whether the filter tracks or localizes from scratch.

3 Experiments

Method

The four sampling choices are interchangeable components of a single filter loop, each selected by name, so a configuration is a set of names and a particle count. The same loop serves both problems; what differs is the state a particle carries, a pose in localization and a pose with its own map in SLAM.

Two simulated worlds are used, both taken unchanged from the reference implementations. The *localization world* is a 10 m square, cyclic at its edges, with four landmarks. The robot takes 30 steps of 0.25 m and 0.02 rad, with motion noise of 0.005 m and 0.002 rad, and measures range and bearing to all four landmarks with noise of 0.2 m and 0.05 rad. Particles start uniformly over the world and all headings, so the opening steps are global localization; the learning experiments add a setting in which they start around the true initial pose, so the filter tracks instead. The *SLAM world* is the FastSLAM scenario of its reference implementation: 500 steps among eight landmarks, observations within 20 m with noise of 0.3 m and 2°, and controls corrupted by noise and a constant yaw-rate offset.

A run is one configuration on one seed, and the seed fixes the world, the trajectory and the filter’s randomness. The same seeds are used for every variant in a comparison, so two variants differ only in the choice being varied. Every configuration is run on twenty seeds, and every table reports the mean and standard deviation over them. A difference is called clear when the 95% confidence intervals of the two means do not overlap, and it is only taken between variants sharing the value of the other varied setting.

What is measured: accuracy, as position RMSE and as ATE after the best rigid alignment, plus map error in SLAM; degeneracy, as the mean effective sample size and the fraction of distinct particles surviving a resampling step; consistency, as the median NEES, about 3 when the particle spread matches the error; cost, as runtime per step; and fit, as the log likelihood (5) of the observations.

Reusing an implementation means checking it. Four errors were found in the upstream code, among them a FastSLAM 2.0 update that never draws from its proposal, and each would have biased a comparison below. They are listed in Appendix A; all results use the corrected code.

Sampling

Two experiments. Each varies one sampling choice alone; the proposal and the number of particles are held fixed throughout, at the motion model and at the counts given below.

Resampling scheme

Setup. The four schemes at 50, 200 and 1000 particles in localization, with the motion-model proposal and resampling at every step so that only the scheme differs (240 runs); and the same four in FastSLAM 1.0 at 10 and 50 particles (160 runs).

Result. In localization the four are equally accurate: at every particle count their errors differ by less than the seed-to-seed spread (Fig. 2). They differ clearly in how many distinct particles survive a resampling step, in the expected order: with 1000 particles systematic keeps 72% distinct, stratified 67%, residual 65% and multinomial 54%, and the order holds at 50 and 200. In FastSLAM that difference reaches the estimate, because a particle that is lost takes its map with it (Table 1): with 10 particles systematic gives an ATE of 0.23 m against 0.32 m for multinomial, and a map error of 0.105 m against 0.139 m. The runtimes differ by an order of magnitude, but they reflect the reused implementations, three of which cost $O(N^2)$ where all four can be $O(N)$.

Does the resampling scheme matter? (localization)

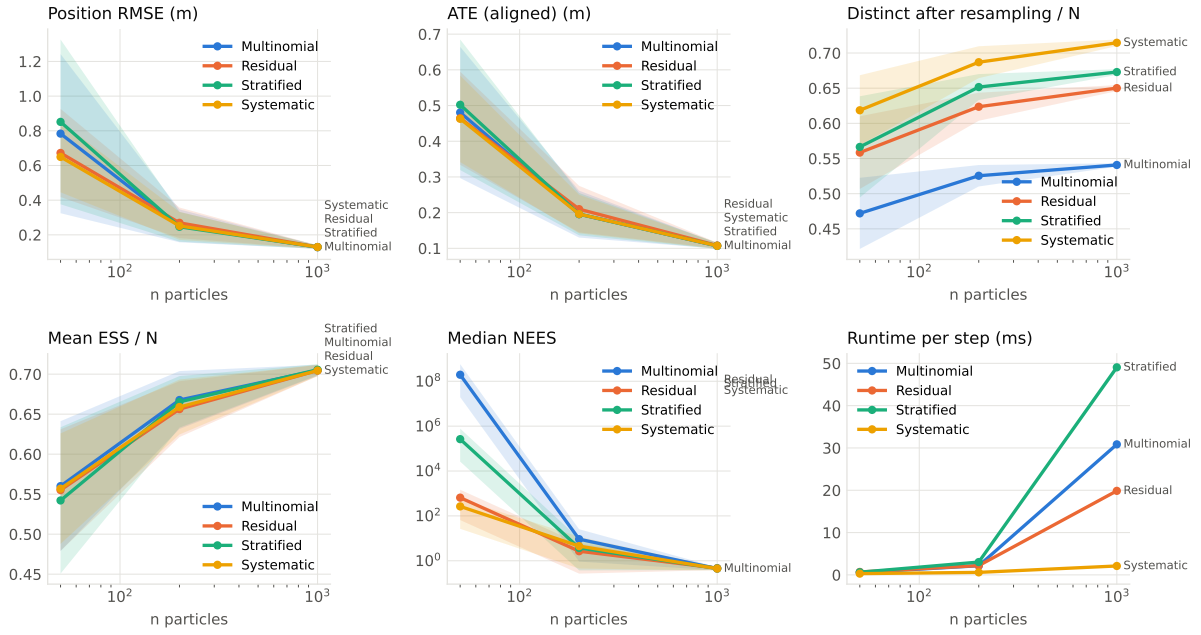


Figure 2: Resampling schemes in localization, against the number of particles. The four are equally accurate; they differ in how many distinct particles survive.

Table 1: Resampling schemes in FastSLAM 1.0, where each particle carries its own map. Mean $\pm$ standard deviation over seeds.

Resampler	Particles	Seeds	Position RMSE (m)	ATE (aligned) (m)	Map error (aligned) (m)	Distinct after resampling / N	Mean ESS / N	Runtime per step (ms)
Multinomial	10	20	0.96 ± 0.55	0.324 ± 0.064	0.139 ± 0.04	0.624 ± 0.0043	0.893 ± 0.0051	1.5 ± 0.012
Multinomial	50	20	0.975 ± 0.47	0.223 ± 0.037	0.0965 ± 0.024	0.61 ± 0.002	0.893 ± 0.0025	2.71 ± 0.01
Residual	10	20	1.02 ± 0.84	0.277 ± 0.048	0.13 ± 0.037	0.785 ± 0.0024	0.892 ± 0.0041	1.57 ± 0.018
Residual	50	20	0.759 ± 0.41	0.209 ± 0.055	0.0832 ± 0.016	0.778 ± 0.0025	0.892 ± 0.0028	2.89 ± 0.02
Stratified	10	20	1.06 ± 0.71	0.247 ± 0.058	0.114 ± 0.037	0.801 ± 0.0042	0.893 ± 0.0043	1.51 ± 0.0073
Stratified	50	20	0.58 ± 0.31	0.205 ± 0.039	0.069 ± 0.02	0.787 ± 0.0016	0.892 ± 0.0021	2.8 ± 0.0088
Systematic	10	20	0.973 ± 0.61	0.233 ± 0.044	0.105 ± 0.032	0.86 ± 0.0038	0.895 ± 0.0037	1.46 ± 0.023
Systematic	50	20	0.879 ± 0.54	0.242 ± 0.1	0.0754 ± 0.016	0.858 ± 0.0019	0.892 ± 0.0024	2.35 ± 0.17

When to resample

Setup. Eight rules in localization with 100 particles, systematic resampling and the motion-model proposal: at every step, never, an ESS threshold of 0.2, 0.5 or 0.8, and a largest-weight threshold of 0.1, 0.2 or 0.5 (160 runs).

Result. Not resampling at all triples the error, 1.39 m against roughly 0.46 m (Fig. 3). Every rule that does resample is equally accurate, whether it acts at every step or only when the effective sample size or the largest weight crosses a threshold. The thresholds change how often resampling happens, between a third of the steps and all of them, not how accurate the filter is; resampling less often leaves each step with more degenerate weights and fewer distinct particles afterwards.

Parameter learning

Four experiments. All of them sweep one noise parameter while the other three are held at the values the world uses, scoring every value by (5). Unless stated otherwise the filter uses the motion-model proposal, systematic resampling, an ESS threshold of 0.5 and 2000 particles, and

When should the filter resample?

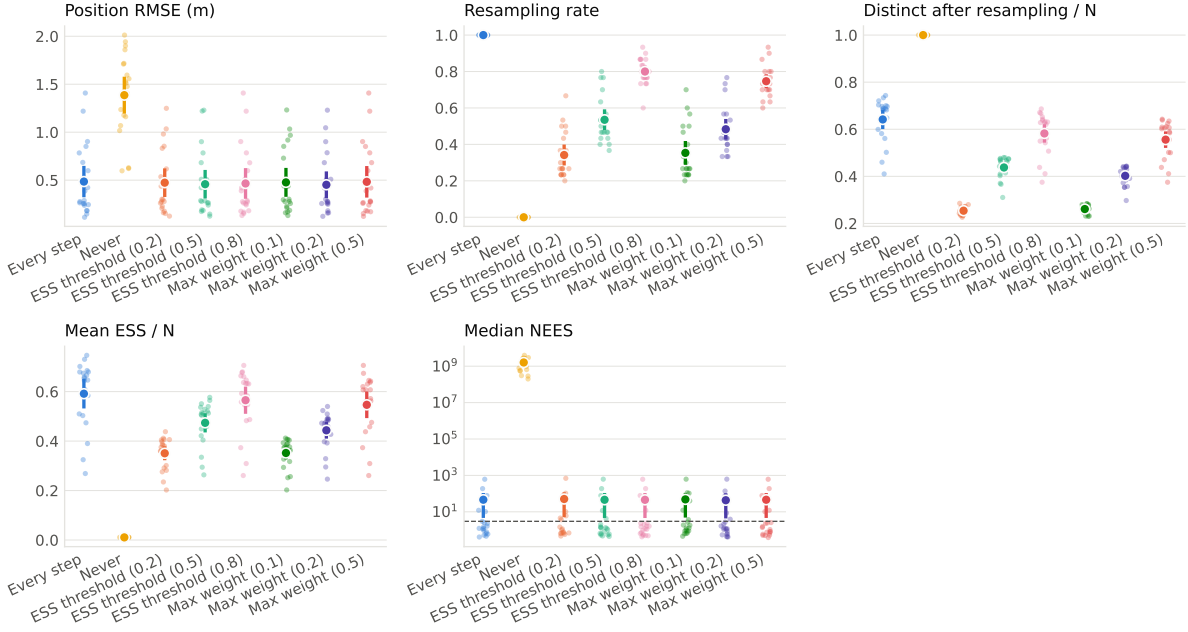


Figure 3: When to resample, in localization with 100 particles. Never resampling stands apart; the rules that do resample are indistinguishable.

starts around the true initial pose.

Recovering the four noise parameters

Setup. Each of the four swept in turn: the range noise over nine values (180 runs), the bearing noise over nine (180 runs), the forward motion noise over seven (140 runs) and the turn noise over seven (140 runs).

Result. The two measurement parameters come back. Both sweeps have a clear maximum at the true value: 0.19m against a true 0.2m (Fig. 4) and 0.047rad against a true 0.05 rad (Table 2), both within one step of the sweep. Too little assumed noise makes the measurements that arrived look impossible and the score falls steeply, from 205 at 0.19m to -252 at 0.08m; too much makes every pose explain them equally well and the score decays slowly, to 98 at 0.8m. The penalty is one-sided, as the model predicts: for a Gaussian residual of true width σ^* scored under σ , the expected score is $-\log \sigma - \sigma^{*2}/2\sigma^2$ up to a constant, maximized at σ^* but falling like σ^{-2} below it and only like $\log \sigma$ above. With 120 range residuals that expression predicts the whole measured curve with nothing fitted: at 0.142, 0.253, 0.45 and 0.8m it gives -18 , -6 , -49 and -110 relative to the peak, against measured -20 , -5 , -47 and -108 . It parts company only at the far left, predicting -205 where -457 is measured, because there the weights degenerate and the estimator's downward bias takes over.

The two motion parameters do not come back. Both sweeps are flat, the log likelihood holding at 205 to 206 while the assumed noise varies by a factor of 30 (Tables 3 and 4). Their true values, 0.005m and 0.002rad, are small next to the measurement noise the filter corrects with at every step, so the observations barely depend on them.

Each seed is a separate recording, so an estimate can also be formed from each one alone. All 20 recordings pick 0.047rad for the bearing noise and 17 of 20 pick 0.19m for the range noise; the 20 estimates of the turn noise spread over the whole sweep. Agreement between recordings is therefore a usable test of whether a parameter is being estimated at all, and it also tests the

tying assumption, since a noise level that changed between runs would scatter even where the likelihood is sharp.

Can the measurement noise be learned from the data? (range)

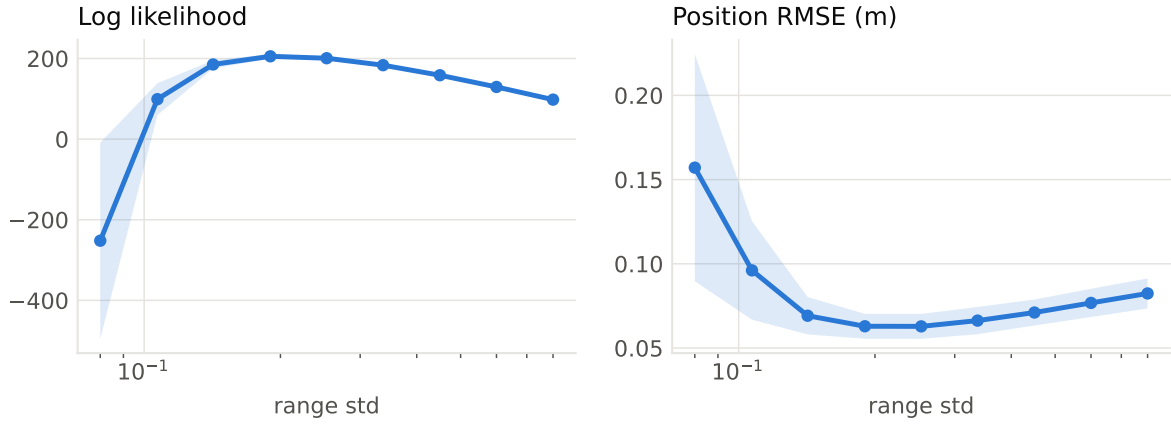


Figure 4: The assumed range noise: the log likelihood peaks at the value the world uses, 0.2 m, and the error is lowest at the same place.

Table 2: Sweeping the assumed bearing noise; the world uses 0.05 rad. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Log likelihood	Position RMSE (m)
0.02	20	-45.8 $\pm$ 1.2e+02	0.126 $\pm$ 0.075
0.0267	20	130 $\pm$ 30	0.0836 $\pm$ 0.031
0.0356	20	190 $\pm$ 15	0.0682 $\pm$ 0.019
0.0474	20	206 $\pm$ 12	0.0626 $\pm$ 0.017
0.0632	20	200 $\pm$ 11	0.0624 $\pm$ 0.017
0.0843	20	181 $\pm$ 10	0.0653 $\pm$ 0.017
0.1125	20	156 $\pm$ 10	0.0698 $\pm$ 0.019
0.15	20	127 $\pm$ 9.9	0.0736 $\pm$ 0.02
0.2	20	95.1 $\pm$ 9.8	0.0779 $\pm$ 0.022

Table 3: Sweeping the assumed forward motion noise; the world uses 0.005 m. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Log likelihood	Position RMSE (m)
0.002	20	206 $\pm$ 12	0.0636 $\pm$ 0.018
0.00356	20	206 $\pm$ 12	0.0638 $\pm$ 0.018
0.00632	20	206 $\pm$ 12	0.063 $\pm$ 0.017
0.01125	20	206 $\pm$ 12	0.0626 $\pm$ 0.016
0.02	20	206 $\pm$ 12	0.0645 $\pm$ 0.016
0.0356	20	205 $\pm$ 12	0.0679 $\pm$ 0.014
0.0632	20	202 $\pm$ 12	0.0771 $\pm$ 0.018

How much data identifiability needs

Setup. The turn sweep repeated on a 400-step trajectory instead of 30, with nothing else changed, at 1000 particles (140 runs).

Table 4: Sweeping the assumed turn noise over 30 steps; the world uses 0.002 rad. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Log likelihood	Position RMSE (m)
0.0008	20	205 $\pm$ 12	0.0637 $\pm$ 0.018
0.00142	20	205 $\pm$ 12	0.0633 $\pm$ 0.017
0.00253	20	206 $\pm$ 12	0.0637 $\pm$ 0.018
0.0045	20	206 $\pm$ 12	0.0629 $\pm$ 0.017
0.008	20	206 $\pm$ 12	0.0632 $\pm$ 0.017
0.0142	20	206 $\pm$ 12	0.0632 $\pm$ 0.015
0.0253	20	205 $\pm$ 12	0.065 $\pm$ 0.014

Result. The flat curve becomes one with a maximum at the true value, and an assumed drift ten times too small is rejected by thousands of nats (Fig. 5). The 20 recordings concentrate again instead of spreading. Identifiability is therefore a property of the data rather than of the method: because the parameters are tied across time slices, the curvature of the likelihood grows with the number of steps, as Section 1 predicts.

Does more data make the motion noise identifiable? (turn, 400 steps)

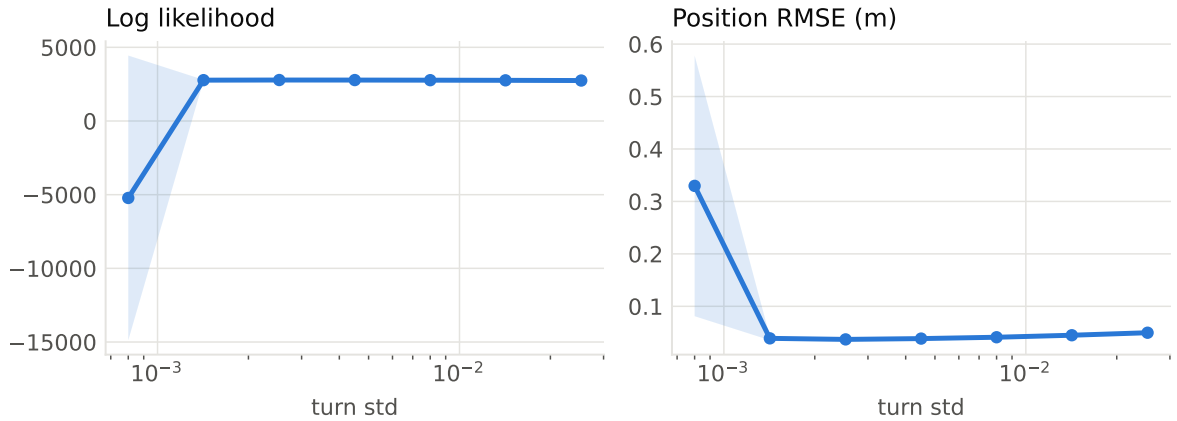


Figure 5: The same turn sweep over 400 steps: enough steps identify a value that 30 steps cannot.

Whether the filter has to be tracking

Setup. The range sweep run twice, once from a known start pose and once from the uniform start used by the sampling experiments (360 runs).

Result. From the uniform start the maximum disappears (Fig. 6). With the true, small noise the filter does not find the robot within 30 steps from a uniform prior: the position error is 2.3 m against 0.063 m from a known start, and the log likelihood is four orders of magnitude lower. The score then keeps improving as the assumed noise is widened, because a wider noise lets the particles spread enough to be useful, and only 2 of 20 recordings agree. What the likelihood reports in this regime is which setting lets this filter work, not which setting generated the data.

Does learning depend on the filter tracking at all?

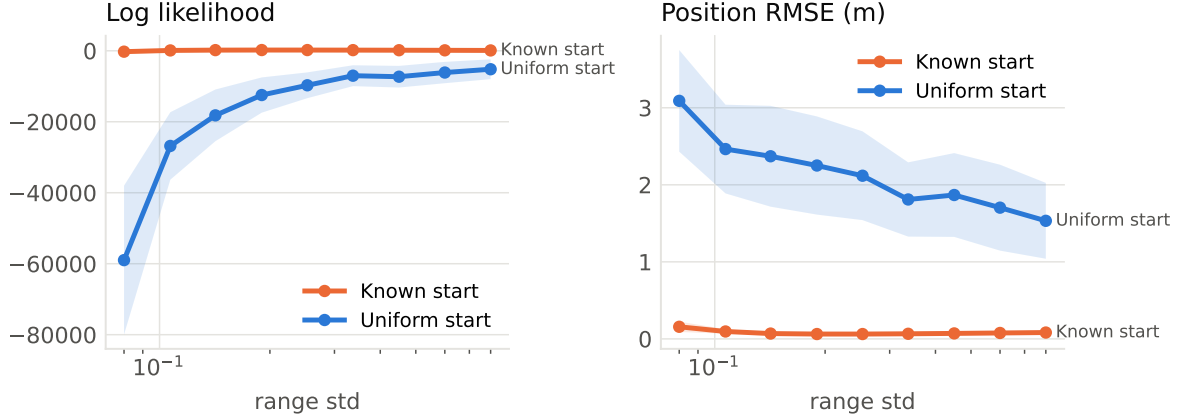


Figure 6: The range sweep from a known start and from the uniform start of the sampling experiments. Note the scale: from the uniform start the estimate is four orders of magnitude lower and has no maximum.

What the estimate needs from the inference

Setup. The range sweep repeated at 25, 100, 500 and 2000 particles (720 runs), and again under each of the four resampling schemes at 200 particles (720 runs).

Result. At the true value the estimated log likelihood rises from 28 ± 316 with 25 particles to 205 ± 13 with 2000, and agreement between recordings rises with it, from none at 25 particles to 17 of 20 at 2000 (Fig. 7): both the bias and the spread shrink as the approximation improves. The resampling scheme makes no difference: at 200 particles all four place the maximum at 0.19 m, with 14 or 15 of 20 recordings agreeing (Table 5).

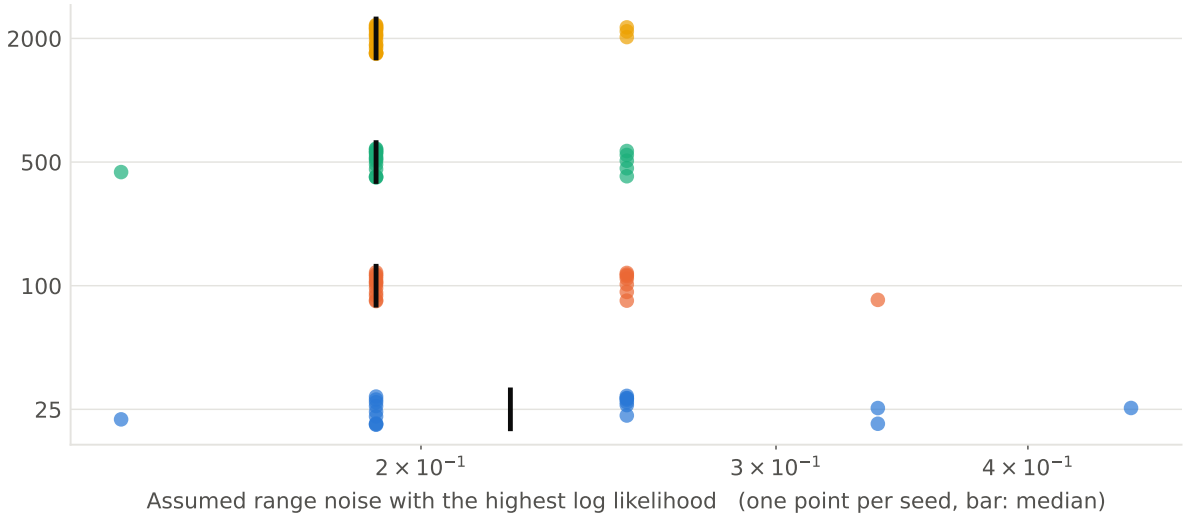


Figure 7: The range noise each recording picks out on its own, against the number of particles.

Table 5: The range noise picked out under each resampling scheme, with 200 particles.

Resampler	Seeds	Best assumed range noise (median)	Range over seeds	Seeds at the median
Multinomial	20	0.19	0.19 to 0.45	15 of 20
Residual	20	0.19	0.19 to 0.337	14 of 20
Stratified	20	0.19	0.19 to 0.253	15 of 20
Systematic	20	0.19	0.19 to 0.337	14 of 20

4 Discussion and conclusion

Which choices mattered. The number of particles and the decision to resample at all decided most of the accuracy. The choices made inside resampling decided particle diversity instead, and the two are not the same thing: a difference in diversity reaches the estimate only when particles are scarce and each one carries a map. The proposal sits between them, mattering most where particles are few and losing its advantage as they are added. A measurement-informed proposal also costs more per particle, so the fair comparison is at equal runtime rather than at equal particle count, and there its advantage is much smaller. Adaptive resampling buys no accuracy in these worlds but saves resampling steps at no cost.

What limits learning. Two limits, of different kinds. The first is the inference: the objective is estimated by the same weights that do the filtering, so a filter that has not found the robot reports which setting lets it work rather than which setting generated the data, and a filter with too few particles reports noise. Inference quality is a precondition for learning, not merely something learning improves. The second is the data: the motion parameters are perfectly well defined and undetermined by 30 steps of this world, and become determined only when the trajectory is long enough for their effect to show against the measurement noise. No amount of computation removes that, and agreement between independent recordings separates the two cases cheaply. The one-sided shape of the likelihood also explains a common practice: since understating a noise level is penalized far more heavily than overstating it, values set by hand are rationally kept large, as the settings inherited from the upstream code are.

On reusing implementations. Reused code has to be checked function by function before it is compared: four errors in the upstream code would each have biased a comparison, one of them leaving resampled particles sharing their state, which would have masked exactly the diversity effect measured here.

Limitations. The worlds are small and simulated, with known correspondences; real sensors, unknown data association and larger maps would make degeneracy and proposal quality more important. Each experiment varies one choice while the others are fixed, so interactions are not measured. The rules for when to resample were compared only in localization, and the resampling schemes in SLAM only with FastSLAM 1.0. The proposal and the number of particles were held fixed rather than varied, so nothing here says how the two choices studied interact with them. The learning experiments estimate one parameter at a time from a known start, so they do not give the joint maximum over all four, and a sweep is a cruder optimizer than expectation maximization.

Conclusion. Localization and SLAM, written as a dynamic Bayesian network, make the same three demands as any graphical model: a structure that justifies a factorization, an inference procedure for what the structure leaves open, and parameters that have to come from somewhere.

This project took the structure as given and worked on the other two. On the inference side it measured which sampling choices change the answer, and found that most of the accuracy is decided before the resampling scheme is chosen. On the learning side it estimated the model’s own noise parameters from recorded runs, recovered the measurement noise to within one step of the sweep, and showed that the motion noise is identifiable only with a longer trajectory. The two sides meet in one result: the quality of the approximate inference sets the ceiling on what can be learned.

With more time the obvious next steps would be MCMC moves after resampling [12], which attack particle impoverishment directly; the rules for when to resample in SLAM; estimating the four parameters together by expectation maximization instead of one sweep at a time; and running the whole thing on real data, where the correspondences are not given.

A Upstream Code and Corrections

The localization filters, resampling schemes and resampling rules come from the particle filter tutorial of Elfring et al. [8] (commit `e6014b7`, MIT); FastSLAM 1.0 and 2.0 from PythonRobotics [9] (commit `1fe4fb9`, MIT). The upstream code is downloaded at these commits and never modified: the schemes and rules are called unchanged, while the models, proposals and FastSLAM updates are copied, vectorized over particles, and moved to log weights. Every correction can be switched off, and with them off the copies match the upstream functions on the same inputs and random numbers; those comparisons are part of the test suite.

Three corrections change a result reported here. The localization likelihood omits the Gaussian’s normalizing constant, which cancels once the weights are normalized but is the term that penalizes a wider assumed noise; without it the marginal likelihood (5) grows without bound and the learning sweeps have no maximum. FastSLAM resampling copies the list of particles but not the particles themselves, so resampled particles share their state and their landmark arrays, which would have hidden the difference between the schemes. And the likelihood compares bearings without wrapping the difference, so a small error across $\pm\pi$ looks like almost 2π . Nine further errors, none of which changes a conclusion, are corrected in the code and listed there.

References

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005. [Online]. Available: <https://mitpress.mit.edu/9780262201629/probabilistic-robotics/>.
- [2] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009. [Online]. Available: <https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/>.
- [3] N. J. Gordon, D. J. Salmond, and A. F. M. Smith, “Novel approach to nonlinear/non-Gaussian Bayesian state estimation,” *IEE Proceedings F (Radar and Signal Processing)*, vol. 140, no. 2, pp. 107–113, 1993. DOI: [10.1049/ip-f-2.1993.0015](https://doi.org/10.1049/ip-f-2.1993.0015).
- [4] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, “A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking,” *IEEE Transactions on Signal Processing*, vol. 50, no. 2, pp. 174–188, 2002. DOI: [10.1109/78.978374](https://doi.org/10.1109/78.978374).
- [5] A. Doucet, N. de Freitas, K. Murphy, and S. Russell, “Rao-Blackwellised particle filtering for dynamic Bayesian networks,” in *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, 2000. arXiv: [1301.3853](https://arxiv.org/abs/1301.3853).
- [6] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit, “FastSLAM: A factored solution to the simultaneous localization and mapping problem,” in *Proceedings of the AAAI National Conference on Artificial Intelligence*, 2002. [Online]. Available: <https://cdn.aaai.org/AAAI/2002/AAAI02-089.pdf>.

- [7] A. Doucet and A. M. Johansen, “A tutorial on particle filtering and smoothing: Fifteen years later,” in *The Oxford Handbook of Nonlinear Filtering*, Oxford University Press, 2011. [Online]. Available: https://www.stats.ox.ac.uk/~doucet/doucet_johansen_tutorialPF2011.pdf.
- [8] J. Elfring, E. Torta, and R. van de Molengraft, “Particle filters: A hands-on tutorial,” *Sensors*, vol. 21, no. 2, p. 438, 2021. DOI: [10.3390/s21020438](https://doi.org/10.3390/s21020438).
- [9] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, *PythonRobotics: A Python code collection of robotics algorithms*, 2018. arXiv: [1808.10703](https://arxiv.org/abs/1808.10703). [Online]. Available: <https://github.com/AtsushiSakai/PythonRobotics>.
- [10] R. Douc, O. Cappé, and E. Moulines, “Comparison of resampling schemes for particle filtering,” in *Proceedings of the International Symposium on Image and Signal Processing and Analysis (ISPA)*, 2005, pp. 64–69. DOI: [10.1109/ISPA.2005.195385](https://doi.org/10.1109/ISPA.2005.195385).
- [11] J. D. Hol, T. B. Schön, and F. Gustafsson, “On resampling algorithms for particle filters,” in *IEEE Nonlinear Statistical Signal Processing Workshop*, 2006, pp. 79–82. DOI: [10.1109/NSSPW.2006.4378824](https://doi.org/10.1109/NSSPW.2006.4378824).
- [12] W. R. Gilks and C. Berzuini, “Following a moving target—Monte Carlo inference for dynamic Bayesian models,” *Journal of the Royal Statistical Society: Series B*, vol. 63, no. 1, pp. 127–146, 2001. DOI: [10.1111/1467-9868.00280](https://doi.org/10.1111/1467-9868.00280).